MMDU MULLANA OFFICIAL SYLLABUS • BCSE-008 • UNIT 4

Unit 4: Advanced Data Handling, Pointers, Structures, DMA & File I/O

Subject: Computational & Problem Solving Using C (BCSE-008) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 45 mins

Address arithmetic, pointer taxonomy, Near, Far, and Huge Pointers (16-bit offset vs 32-bit segment:offset and auto-normalization), Struct vs Union memory alignment, Dynamic Memory Allocation (malloc, calloc, realloc, free), and File I/O streams.

1. Pointer Taxonomy: Wild, Dangling, Void & Function Pointers

1. Advanced Pointer Classification Matrix

Pointer Type	Syntax & Definition	Memory Hazard / Behavior	Remediation / Safe Practice
Wild Pointer	`int ptr;` *(Uninitialized)*	Points to arbitrary memory; dereferencing corrupts RAM or crashes process.	Always initialize to `NULL`: `int ptr = NULL;`.
Dangling Pointer	`free(ptr);` (Target freed)	Points to deallocated storage that may be reassigned to other subsystems.	Set pointer to `NULL` immediately after freeing: `free(ptr); ptr = NULL;`.
Void Pointer	`void vptr = &x;`	Generic memory pointer without data type association. Cannot be directly dereferenced.	Explicitly typecast before dereferencing: `(int)vptr`.
Function Pointer	`int (fp)(int, int) = &add;`	Holds entry address of executable code in text segment. Enables runtime callbacks.	Invoked via `int res = fp(10, 20);`.
Pointer to Array	`int (p)[5];`	Single pointer referencing an entire array block of 5 integers.	Used in multi-dimensional array parameter passing.

| **Array of Pointers** | `int p[5];` | Array containing 5 independent integer pointer variables. | Used in ragged 2D tables and string arrays (`char *argv[]`). |

2. Real-Mode 16-Bit Architecture: Near, Far, and Huge Pointers

8086 Segmented Memory Pointer Architecture

In 16-bit real-mode architecture (such as the 8086 microprocessor with 20-bit address bus addressing 1 MB of RAM), pointers are classified into three types:

> $$extPhysicalAddress = (\text{Segment Register} \times 16) + \text{Offset}$$

Pointer Classification	Address Width	Internal Structure	Accessible Memory Span	Key Characteristics & Comparison Rules
Near Pointer	**16-bit (2 Bytes)**	Offset only	**64 KB** (within default DS/CS)	Fastest address arithmetic; cannot access data outside the current 64 KB segment.
Far Pointer	**32-bit (4 Bytes)**	16-bit Segment : 16-bit Offset	**1 MB** (Full real-mode RAM)	Can address any byte in 1 MB. **Hazard:** Addresses are unnormalized. `0x1000:0x0010` and `0x1001:0x0000` point to the identical byte, but `p1 == p2` evaluates to FALSE!

| **Huge Pointer** | **32-bit (4 Bytes)** | Normalized Segment : Offset | **1 MB** (Full real-mode RAM) | **Auto-Normalizing:** Any offset ≥ 16 is converted: $extSegmentleftarrow\text{Segment} + (\text{Offset}/16)$ and $extOffsetleftarrow\text{Offset}pmod16$. Relational comparisons are mathematically accurate across segments. |

3. Structures, Unions & Memory Alignment Architecture

Structure vs. Union Architecture

Dimension	Structure (`struct`)	Union (`union`)
Keyword	`struct Record { int id; char grade; double score; };`	`union Record { int id; char grade; double score; };`
Memory Allocation	Sum of all member sizes + structure padding bytes.	Space equal only to the size of its largest member.
Memory Sharing	Each member has its own distinct, sequential address offset.	**All members share the exact same starting memory address.**
Active Members	All members can store valid values simultaneously.	Only **ONE** member can hold a valid value at any given instant.
Size Calculation	$extsizeof(\text{struct}) = 4 + 1\ (\text{pad } 3) + 8 = mathbf16$ Bytes	$extsizeof(\text{union}) = max(4, 1, 8) = mathbf8$ Bytes

| **Member Access** | `var.id` (Direct) or `ptr->id` (Indirect via pointer) | `u.id` (Direct) or `u_ptr->id` (Indirect via pointer) |

4. Dynamic Memory Allocation (DMA) Complete Lifecycle

The Complete DMA Lifecycle

```
1. Allocation (malloc / calloc)
      |
      ▼
2. NULL Guard Check (if ptr == NULL -> exit)
      |
      ▼
3. Reallocation (realloc if size changes)
      |
      ▼
4. Safe Deallocation (free(ptr) -> ptr = NULL)
```

DMA Functions ( ):

- `malloc(size_t size)` : Allocates raw, uninitialized bytes from the heap pool (contains garbage).
- `calloc(size_t n, size_t size)` : Allocates memory and initializes all bits to zero.
- `realloc(void *ptr, size_t new_size)` : *Resizes an existing heap block without losing existing data.*
- `free(void ptr)` : Returns memory back to the heap pool to prevent memory leaks.

5. File Handling Operations & Stream Modes

File Stream Modes & I/O Functions

Mode	Stream Meaning	If File Missing	If File Already Exists	Stream Functions
`"r"`	Read Text	Returns `NULL` error pointer	Opens file; read position at byte 0	`fscanf()` , `fgetc()` , `fgets()`
`"w"`	Write Text	Creates a new empty file	**Truncates (erases) file to 0 bytes!**	`fprintf()` , `fputc()` , `fputs()`
`"a"`	Append Text	Creates a new empty file	Preserves data; all writes appended at EOF	`fprintf()` , `fputs()`
`"r+"`	Read & Write	Returns `NULL` error pointer	Opens file for update without truncation	`fseek()` , `ftell()` , `rewind()`

| `"rb"` / `"wb"` | Binary Stream | Opens file for binary mode | No CR/LF translation; raw byte storage | `fread()` , `fwrite()` |

6. University Examination Solved Questions (Unit 4)

University Solved Examination Problems (MMDU Pattern)

Question 10 (1 Mark): Dangling Pointer Remediation

Question: What hazard is introduced after calling `free(p);`, and how is it resolved?

Step-by-Step Solution:

```
Pointer p ➔ [ Address 0x5000 ] ➔ Heap block released ➔ Pointer p still retains 0x5000
```

`p` becomes a **Dangling Pointer**. Dereferencing it leads to memory corruption or crashes. Remediation: explicitly assign `p = NULL;` immediately after calling `free(p);`.

Question 20 (4 Marks): Near, Far, and Huge Pointers in 8086

Question: Explain Near, Far, and Huge pointers in 16-bit 8086 architecture with segment arithmetic.

Step-by-Step Solution:

1. **Near Pointer:** 16-bit offset. Limited to current 64 KB segment. Fast arithmetic.

2. **Far Pointer:** 32-bit (16-bit segment : 16-bit offset). Addresses full 1 MB. Unnormalized: `0x1000:0x0010` and `0x1001:0x0000` point to the same physical byte, but relational equality evaluates to false!

3. **Huge Pointer:** 32-bit auto-normalized pointer. Converts any offset ≥ 16: $ext Segment left tarrow \text{Segment} + (\text{Offset}/16)$ and $ext Offset left tarrow \text{Offset} p mod 16$. Relational comparisons are mathematically accurate.

Question 23 (6 Marks): Student Records with DMA and Binary File I/O

Question: Design an architecture in ANSI C to dynamically allocate an array of `Student` structures on the heap, write records to `students.dat`, and read them back.

Step-by-Step Solution:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int rollNo;
    char name[32];
    float gpa;
} Student;

void processRecords(int n) {
    // 1. Dynamic Allocation
    Student *list = (Student*)malloc(n * sizeof(Student));
    if (!list) { printf("Heap allocation failed!
"); return; }

    // 2. Initialize sample record
    list[0].rollNo = 101; sprintf(list[0].name, "Aman"); list[0].gpa = 8.85f;

    // 3. Persist to Binary File
    FILE *fp = fopen("students.dat", "wb");
    if (fp) {
        fwrite(list, sizeof(Student), n, fp);
        fclose(fp);
    }

    // 4. Read back into fresh heap buffer
    Student *readList = (Student*)malloc(n * sizeof(Student));
    fp = fopen("students.dat", "rb");
    if (fp) {
        fread(readList, sizeof(Student), n, fp);
        printf("Recovered: Roll %d, Name: %s, GPA: %.2f
",
               readList[0].rollNo, readList[0].name, readList[0].gpa);
        fclose(fp);
    }

    // 5. Clean up heap memory
    free(list); free(readList);
}
```

Question 24 (6 Marks): Function Pointers & Callback Sorting

Question: Explain the mechanism of Function Pointers in C by implementing a generic bubble sort that accepts a comparison callback.

Step-by-Step Solution:

```c
#include <stdio.h>

int ascending(int a, int b) { return a > b; }
int descending(int a, int b) { return a < b; }

void genericSort(int arr[], int n, int (*compare)(int, int)) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (compare(arr[j], arr[j + 1])) { // Dynamic callback!
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int data[] = {45, 12, 89, 34};
    genericSort(data, 4, ascending);  // Sorts [12, 34, 45, 89]
    genericSort(data, 4, descending); // Sorts [89, 45, 34, 12]
    return 0;
}
```